

Задача № 1987. Number of Unique Good Subsequences

Вам дана двоичная строка `binary`. Подпоследовательность `binary` считается хорошей, если она не пустая и в ней нет ведущих нулей (за исключением "0").

Найдите количество уникальных хороших подпоследовательностей в `binary`.

Например, если `binary = "001"`, то все хорошие подпоследовательности — это ["0", "0", "1"], поэтому уникальные хорошие подпоследовательности — это "0" и "1". Обратите внимание, что подпоследовательности "00", "01", и "001" не являются хорошими, потому что в них есть ведущие нули.

Верните количество уникальных хороших подпоследовательностей из `binary`. Поскольку ответ может быть очень большим, верните его по модулю $10^9 + 7$.

Подпоследовательность — это последовательность, которая может быть получена из другой последовательности путем удаления некоторых элементов или без удаления элементов, при этом порядок оставшихся элементов не меняется.

Алгоритм

Задача решается с использованием динамического программирования с хранением двух состояний:

1. `z_end` — кол-во верных последовательностей, оканчивающихся на 0
2. `o_end` — кол-во верных последовательностей, оканчивающихся на 1

При обработке каждого символа строки обновляем эти значения:

1. Если символ 0:

Все последовательности, которые оканчиваются на 1, могут стать последовательностями, оканчивающимися на 0 (добавляем 0 в конец)

Формула: $z_end = (z_end + o_end) \% MOD$

Устанавливаем флаг `f = true`, обозначающий, что в строке есть хотя бы один ноль

2. Если символ 1:

Все последовательности, которые оканчиваются на 0 и 1, могут стать вероятностями, оканчивающимися на 1

Плюс можно создать новую последовательность из одной единицы (поэтому +1)

Формула: $o_end = (z_end + o_end + 1) \% MOD$

В конце находим сумму $(z_end + o_end)$ и добавляем 1, если в строке были нули.

Лемма 1 (Корректность переходов)

При добавлении нового символа к существующим последовательностям мы не теряем ни одной верной последовательности и не учитываем лишние.

Доказательство:

Для 0: все последовательности, оканчивающиеся на 1, после добавления 0 будут оканчиваться на 0. Последовательности, уже оканчивающиеся на 0, остаются без

изменений (не удваиваются, так как добавление 0 к ним дало бы две 0 подряд, что, видимо, запрещено условием).

Для 1: все существующие последовательности (как на 0, так и на 1) после добавления 1 будут оканчиваться на 1. Также добавляем новую последовательность из одной 1.

Лемма 2 (Учёт всех последовательностей)

Флаг f необходим для корректного подсчёта последовательностей, начинающихся с 0.

Доказательство:

Если в строке вообще нет нулей, то последовательности, содержащие 0, невозможны. Добавление +1 в конце учитывает случай, когда вся строка состоит из единиц, но при этом есть последовательности, начинающиеся с нуля (которых быть не может). Корректировка флагом f исправляет этот момент.

Асимптотика (время)

Алгоритм проходит по строке ровно один раз:

- Длина строки: n
- На каждом шаге выполняется $O(1)$ операций

Общая сложность: $O(n)$

Асимптотика (память)

Алгоритм использует:

- Строка длины n
- Целочисленные переменные: z_end , o_end , f
- Константная память для вспомогательных переменных

Общие затраты памяти: $O(n)$ на хранение строки $O(1)$ дополнительно, то есть $O(n)$